

Final Project - Jared Krug

<u>Identify the vulnerability</u>	<u>What harm could it do?</u>	<u>How to fix it</u>
1. The user 'backdoor' has access to the root-level meaning they can do anything they want. I found this by looking in /etc/passwd and looking at the permissions of the user. The user 'toor' also seems to be a root user with a weak password Steps: 1.cat /etc/passwd 2.understand user 'backdoor' has root privileges (sudo)	Because this user has full access, they can do just about anything they want such as installing harmful things, deleting important information, stealing data, or just overall destroying the system as a whole.	To fix this, I would probably disable the user or even delete them by using a delete command. Another step to take might be to change the passwords to all important accounts, just in case the user has already been used to steal information. To be safe, it might also be useful to look into potential mechanisms that allow for someone like this to get back in and prevent them from working somehow.
2. Unencrypted communication for finger (port 79), http (port 80), ftp (port 21), telnet (Port 23), and smtp (port 25). These were found by checking the internet connections via -tulnp. Steps: 1.sudo netstat -tulnp 2.see that there are plaintext protocols in use (ftp for example) 3.realize not secure	This is a problem because an attacker will be able to use Wireshark, for example, to see usernames, passwords, and other important information because everything is being sent via a plaintext protocol, which is incredibly insecure. With the telnet one, attackers are able to intercept/hijack login sessions.	The surefire way to fix these would be to simply not use them and instead use something like SFTP (port 22) in place of FTP, and HTTPS (port 443) in place of HTTP. These are more secure protocols that make use of things such as ssh in order to encrypt private information. For everything else, it'd be best to set up SSL/TLS with everything (apache, postfix, etc).
3. There is a problem with file permissions that was found in /etc/shadow allowing all users to read the file. Steps: 1.ls -l /etc/shadow 2.see that the permissions are not properly configured	The file shows: -rw-r--r-- This is a problem because it means that all users can read the contents of the file. This is a huge vulnerability because now all of the users' hashed passwords are open to be copied and cracked allowing for an attacker to have unauthorized access to the user accounts.	The best fix, I think, is to modify the permissions of the file so that only the root can read and write, and the shadow group can read (pretty sure this is the basic configuration for this type of file). Command: chmod 640 /etc/shadow This command makes it so that the file permissions look like this: -rw-r-----
4. The webpage on http is vulnerable to SQL injection	The login functionality of the web application allows anyone to	It'd be best to use prepared statements that are able to safely

because of its lack of user authentication. Steps: 1.ssh into backdoor on different device 2.use curl to look at html 3.login with weak credentials (given in lab before) 4.attempt script used in lab before	bypass authentication and can be any user they want to be on the system. A bad user can set up a script to execute on the system to gain any information they'd like. This could steal the users information such as their cookies as well as give the bad user access to that user's account.	handle user input as well as sanitize the inputs it gets from the login page. On top of that, it'd make sense to prevent things such as SQL commands, any type of quotations, logic statements (OR for example), as well as any other characters that can be easily used to get in with little authentication or use scripts ('<', '>', '/', etc).
5. When looking into the sshd_config file, it allows for the ability to use root login as well as empty passwords. Steps: 1.sudo nano /etc/ssh/sshd_config	Within the ssh config file, there are lines that permit root login as well as empty passwords. This is a major issue as it's not only insecure, but it allows for people to have basically unlimited access and have an easy way onto the machine.	The simplest fix would be to edit the sshd_config to use more secure settings/options. The following are what I would change the bad settings to: PermitEmptyPasswords no PermitRootLogin no
6. The FTP service allows anonymous login with no password. With this, the anonymous user can upload files to the vulnerable device that are then assigned root ownership. Steps: 1.login in with anonymous user in FileZilla 2.add a test file into one of the directories (I chose /hacks_here/test/) 3. Checked access within both FileZilla and the ssh'd user.	This is bad because an attacker with FTP access can really easily plant files that appear to originate from root but are really from an unauthenticated anonymous user. So any user can upload files and since it's assigned to root, it can only be removed by root or sudo (I'm pretty sure).	Some necessary fixes include removing the ability to gain FTP access anonymously, ensure that newly uploaded files are not assigned to the root, and set up a way to log unauthorized uploads. After further investigation, the specific file: /etc/vsftpd.conf can be edited to change these settings: anonymous_enable NO anon_upload_enable NO anon_mkdir_write_enable NO
7. One of the open ports (Port 1337 (leet)) seems to listen on all interfaces as well as accept commands. This basically means that any line of text sent to this port will be executed as a bash command with root privileges without authentication. Steps: 1.sudo netstat -tulpn on vulnerable device 2.nc <ip> 1337 on outside device on network	This allow for unauthenticated users to have full root-level remote command execution. This allows for things such as file manipulation, privilege escalation, etc. This is basically just as dangerous as whoever has access to the 'backdoor' user, except it can be done remotely, so in short this seems like an incredibly bad vulnerability.	The fix I could think of for this would be to block access with a firewall so that remote/external access is not able to persist, but a more surefire fix would be to kill the process and then remove it from any startup scripts so that it cannot be accessed. You can do this by editing the following: Locate the issue (evil.service) systemctl disable evil.service rm /lib/systemd/system/evil.service

3.run 'whoami' and see root. run 'touch /tmp/test' and the vulnerable device shows a file in the root directory

Please draw a network diagram illustrating the comprehensive network infrastructure established over the semester. This diagram should resemble Lab 4's layout but should provide a thorough depiction of the network, including each configured machine placed appropriately with their respective services, ports, and IP addresses indicated.

- All network resources must be assigned an IP address.
- Any necessary port forwarding should be annotated on the Firewall/NAT (excluding UFW firewall rules).
- The LDAP workstation and the Kali Box do not need to be included.
- Ensure the inclusion of virtual IP port forwarding within the diagram.

Insert diagram below:

